

Working with Time and Date Data

What's Covered

1. Why Datetime Features Need Special Treatment
2. pandas datetime — Parsing and Converting
3. Extracting Basic Date and Time Components
4. Derived Features — Age, Duration, Days Since
5. Cyclical Features — Encoding Circular Time
6. Working with Time Zones
7. Handling Date Strings in Different Formats
8. Using Datetime in a ML Pipeline
9. Common Pitfalls
10. Summary

1. Why Datetime Features Need Special Treatment

Dates and timestamps are among the richest sources of predictive signal in real-world datasets — retail sales spike on weekends, fraud occurs more often at odd hours, loan defaults are more common for older accounts. However, raw datetime values like '2024-03-15 14:32:00' are useless to a machine learning model in their raw form.

ML models cannot work with date strings or even raw datetime objects directly. They need numbers. Feature engineering on datetime columns means extracting the useful numeric signals hidden inside — hour, day of week, month, age, duration, is_weekend, etc. — and encoding them in a way the model can use.

1.1 Datetime Features Are Common In:

- E-commerce: order date, delivery date, last purchase date
- Banking: account open date, transaction timestamp, loan start date
- Healthcare: date of birth, appointment date, prescription date
- IoT / Sensors: timestamp of every reading
- HR / Payroll: hire date, resignation date, last promotion date

2. pandas datetime — Parsing and Converting

The first step is always to convert a date column from a raw string (object dtype) into a proper pandas datetime type. Only after conversion can you extract components from it.

```
import pandas as pd

# Raw dataframe with date stored as a string (object type)
df = pd.DataFrame({'order_date': ['2024-01-15', '2024-03-22', '2024-07-04']})

# Check current dtype – will show 'object' (plain string)
print(df['order_date'].dtype)
# object

# Convert to datetime using pd.to_datetime()
df['order_date'] = pd.to_datetime(df['order_date'])

# Now dtype is datetime64[ns]
print(df['order_date'].dtype)
# datetime64[ns]
```

Python

2.1 Specifying Format for Faster and Safer Parsing

```
# If dates are in a specific format, specify it explicitly
# This is faster and avoids pandas guessing wrong format
df['order_date'] = pd.to_datetime(df['order_date'], format='%Y-%m-%d')

# Common format codes:
# %Y = 4-digit year, %y = 2-digit year
# %m = month (01-12), %d = day (01-31)
# %H = hour (00-23), %M = minute, %S = second
# %A = weekday name, %B = month name

# If some dates can't be parsed, use errors='coerce' to produce NaT instead of crash
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')
```

Python

3. Extracting Basic Date and Time Components

Once the column is datetime, pandas provides the .dt accessor to extract any component as a new numeric column.

Attribute	What It Returns
df['date'].dt.year	4-digit year as integer — e.g. 2024
df['date'].dt.month	Month as integer 1–12
df['date'].dt.day	Day of month as integer 1–31
df['date'].dt.hour	Hour as integer 0–23
df['date'].dt.minute	Minute as integer 0–59

Attribute	What It Returns
<code>df['date'].dt.second</code>	Second as integer 0–59
<code>df['date'].dt.dayofweek</code>	Day of week: 0=Monday ... 6=Sunday
<code>df['date'].dt.day_name()</code>	Day name as string: 'Monday', 'Tuesday', ...
<code>df['date'].dt.month_name()</code>	Month name as string: 'January', 'February', ...
<code>df['date'].dt.quarter</code>	Quarter as integer: 1, 2, 3, or 4
<code>df['date'].dt.dayofyear</code>	Day number within the year: 1–365
<code>df['date'].dt.weekofyear</code>	Week number within the year: 1–52
<code>df['date'].dt.is_month_end</code>	Boolean: True if this is the last day of the month
<code>df['date'].dt.is_quarter_end</code>	Boolean: True if this is the last day of a quarter

Python

```
# Extract multiple components at once into new columns
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month
df['day'] = df['order_date'].dt.day
df['day_of_week'] = df['order_date'].dt.dayofweek
df['hour'] = df['order_date'].dt.hour
df['is_weekend'] = df['order_date'].dt.dayofweek.isin([5, 6]).astype(int)

# Drop the original datetime column before feeding to model
df.drop(columns=['order_date'], inplace=True)
```

4. Derived Features — Age, Duration, Days Since

Some of the most powerful datetime features are derived by computing differences between two date columns or between a date column and a reference point (like today or the event date).

4.1 Age of a Record (e.g. Customer Age, Account Age)

Python

```
# Age in years from date of birth to today
today = pd.Timestamp('today')
df['age_years'] = (today - df['dob']).dt.days // 365

# Account age in days from account opening date
df['account_age_days'] = (today - df['account_open_date']).dt.days
```

4.2 Duration Between Two Dates

Python

```
# Duration of a project or loan in days
df['duration_days'] = (df['end_date'] - df['start_date']).dt.days

# Duration in hours for precise timestamps
df['duration_hours'] = (df['end_ts'] - df['start_ts']).dt.total_seconds() / 3600

# Days since last purchase (recency feature — common in marketing)
df['days_since_last_buy'] = (today - df['last_purchase_date']).dt.days
```

4.3 Time-Based Binary Flags

Python

```
# Is this a business day? (Mon-Fri = 1, Sat-Sun = 0)
df['is_weekday'] = (df['date'].dt.dayofweek < 5).astype(int)

# Is this a peak hour? (8am-10am or 5pm-7pm rush hour)
df['is_rush_hour'] = df['timestamp'].dt.hour.isin([8,9,17,18]).astype(int)

# Is this a month-end date? (often important in finance)
df['is_month_end'] = df['date'].dt.is_month_end.astype(int)
```

5. Cyclical Features — Encoding Circular Time

Time features like hour (0–23), day of week (0–6), and month (1–12) are CIRCULAR — hour 23 is very close to hour 0, but if you feed raw integers to a model, it will think 23 and 0 are far apart. This is called the cyclical encoding problem.

Solution: Encode circular features using sine and cosine transformations. This maps every value onto a circle, so 23 and 0 become adjacent points — exactly as they should be.

Python

```
import numpy as np

# Cyclical encoding for HOUR (range 0-23, period=24)
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)

# Cyclical encoding for DAY OF WEEK (range 0-6, period=7)
df['dow_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['dow_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)

# Cyclical encoding for MONTH (range 1-12, period=12)
df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)

# Drop the raw integer columns after encoding
df.drop(columns=['hour', 'day_of_week', 'month'], inplace=True)
```

Why both sin AND cos? A single sine value is ambiguous — $\sin(30^\circ) = \sin(150^\circ)$, so two different times map to the same value. Together, sin and cos uniquely identify every position on the circle.

6. Working with Time Zones

Global datasets often contain timestamps in different time zones. Always convert to a single standard time zone (typically UTC) before extracting features, otherwise hour-of-day features will be meaningless.

Python

```
# Localize a naive (no timezone) timestamp to UTC
df['ts_utc'] = df['timestamp'].dt.tz_localize('UTC')

# Convert from one timezone to another
df['ts_ist'] = df['ts_utc'].dt.tz_convert('Asia/Kolkata')

# Remove timezone info (make naive again) – useful for saving to csv
df['ts_naive'] = df['ts_ist'].dt.tz_localize(None)
```

7. Handling Date Strings in Different Formats

Raw String	Format Code	Example
'15-01-2024'	%d-%m-%Y	Day-Month-Year with dashes
'01/15/2024'	%m/%d/%Y	US format Month/Day/Year
'Jan 15, 2024'	%b %d, %Y	Abbreviated month name
'January 15 2024'	%B %d %Y	Full month name
'2024-01-15 14:30:00'	%Y-%m-%d %H:%M:%S	ISO format with time
'15/01/24'	%d/%m/%y	2-digit year

Python

```
# Parse dates with non-standard formats
df['date'] = pd.to_datetime(df['date_str'], format='%d-%m-%Y')

# If format is mixed/unknown, let pandas infer (slower but flexible)
df['date'] = pd.to_datetime(df['date_str'], infer_datetime_format=True)

# Handle Unix timestamps (seconds since 1970-01-01)
df['date'] = pd.to_datetime(df['unix_ts'], unit='s')
```

8. Using Datetime in a ML Pipeline

Because sklearn's ColumnTransformer and Pipeline do not natively understand datetime types, datetime feature extraction must happen BEFORE data enters the Pipeline — typically as a preprocessing step on the DataFrame itself.

Python

```
# Step 1: parse and extract all datetime features BEFORE the pipeline
def extract_datetime_features(df, col):
    df[col] = pd.to_datetime(df[col])
    df[f'{col}_year'] = df[col].dt.year
    df[f'{col}_month'] = df[col].dt.month
    df[f'{col}_day'] = df[col].dt.day
    df[f'{col}_dayofweek'] = df[col].dt.dayofweek
    df[f'{col}_hour'] = df[col].dt.hour
    df[f'{col}_is_weekend'] = df[col].dt.dayofweek.isin([5,6]).astype(int)
    # Cyclical encoding for month and hour
    df[f'{col}_month_sin'] = np.sin(2*np.pi*df[col].dt.month/12)
    df[f'{col}_month_cos'] = np.cos(2*np.pi*df[col].dt.month/12)
    df.drop(columns=[col], inplace=True)
    return df

# Step 2: apply the function before train-test split
df = extract_datetime_features(df, 'order_date')

# Step 3: now X contains only numeric columns – feed into Pipeline normally
X = df.drop(columns=['target'])
y = df['target']
```

FunctionTransformer from sklearn.preprocessing can also wrap a datetime extraction function into a Pipeline step, but pre-processing the DataFrame before splitting is simpler and equally valid for most projects.

9. Common Pitfalls

- ■ **Feeding raw datetime or date strings into a model:** Models cannot understand dates as text or datetime objects. Always extract numeric components first.
- ■ **Treating hour/month/day-of-week as plain integers:** These are circular — hour 23 and hour 0 are adjacent. Use sin/cos cyclical encoding so the model understands the circular structure.
- ■ **Computing age or duration on the FULL dataset before splitting:** If 'today' or the reference date is fixed, this is fine. But if the reference is computed from the data (like max date), compute it only from the training set to avoid data leakage.
- ■ **Ignoring time zones:** Mixing UTC, IST, and EST timestamps in one column makes hour-of-day features meaningless. Always standardise to one timezone first.
- ■ **Not handling NaT (Not a Time) values:** Failed datetime parses produce NaT (the datetime equivalent of NaN). Use `errors='coerce'` to produce NaT and then handle it like a missing value.
- ■ **Keeping the original datetime column after extraction:** The raw datetime column is not numeric and will cause errors in sklearn. Always drop it after extracting all needed components.

10. Summary

Concept	Key Takeaway
Why extract features?	Models need numbers — raw datetime strings are useless to them
Parse datetime	<code>pd.to_datetime(col, format='...')</code> — convert string to datetime64
.dt accessor	<code>df['col'].dt.year / .month / .day / .hour / .dayofweek</code> etc.
is_weekend	<code>dt.dayofweek.isin([5,6]).astype(int)</code> — 1 for Saturday/Sunday
Age / Duration	<code>(today - df['date']).dt.days</code> — difference between two dates in days
Cyclical encoding	sin and cos of $(2\pi \times \text{value} / \text{period})$ — handles circular time
Why sin AND cos?	Single sin is ambiguous — together they uniquely identify every point on a circle
Time zones	<code>tz_localize()</code> to attach, <code>tz_convert()</code> to change — always standardise to one zone
NaT	Not a Time — the datetime equivalent of NaN; handle like a missing value
Pipeline	Extract datetime features before Pipeline — drop original column after extraction
<code>errors='coerce'</code>	Converts unparseable dates to NaT instead of raising an error

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 31 — Working with Time and Date Data